

《数据库系统原理》报告

数据库系统原理作业 4

张鲈泮

北京交通大学计算机科学与技术学院 AI2201 班 22281052@bjtu.edu.cn

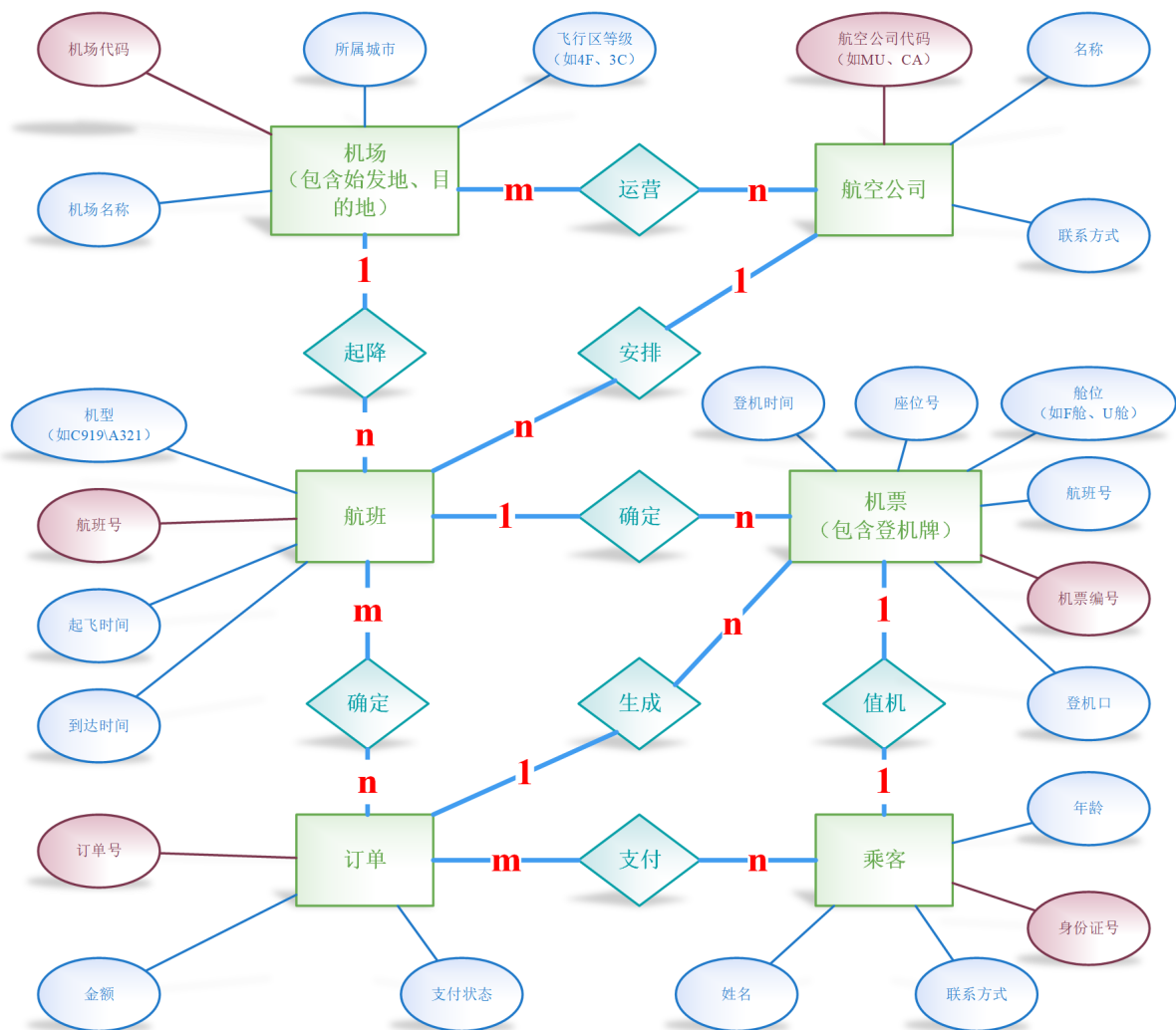


图 1: 航空公司航班查询业务模型 E-R 图, 来自于作业 2

1 题目 1: 视图设计与操作

题目要求: 请针对你在作业 2 中所设计的用户界面的数据需求

1.1 创建视图

至少设计并创建行列子集视图、带表达式的视图和分组视图各一个，并将代码加入自己的建库脚本中。

作业2中所设计的用户界面模式如下（如图2与图3）。

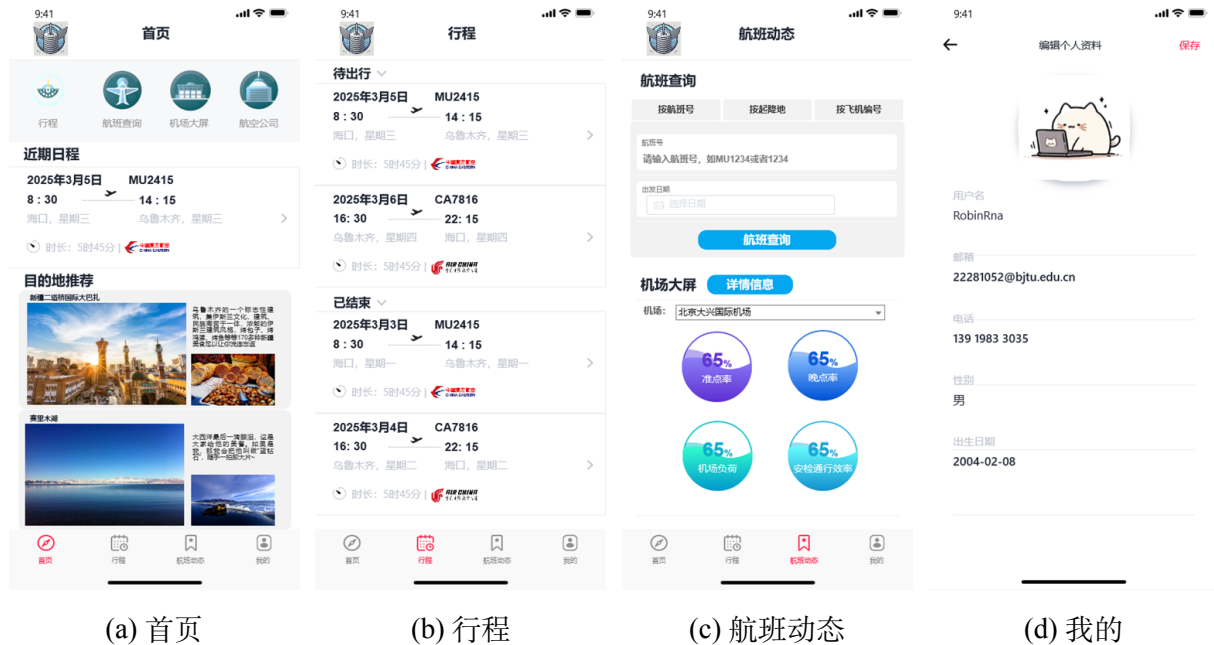


图 2: UI 主要界面



图 3: UI 次要界面

通过图2与图3，可以总结出用户界面的数据需求如下：

1. **用户登录与注册**：用户需要通过手机号码或邮箱进行身份验证，并且需要查看和更新个人信息，包括用户名、邮箱、联系方式、性别和出生日期等。

2. **航班查询与动态**: 用户通过航班号、起降地、机型等条件查询航班信息, 并且显示航班的准点率、延误率等统计数据。
3. **航班信息**: 用户可以查询和展示各个航班的具体信息, 例如航班号、起降时间、航空公司等。并且界面提供用户的历史航班记录, 并展示未来航班的详细信息。
4. **我的个人信息**: 用户能够查看和编辑自己的个人资料, 如用户名、联系方式、性别等。

视图 (View) 是数据库中的一种虚拟表, 它由 SELECT 查询生成, 并允许用户像操作普通表一样操作视图。与普通表不同, 视图不存储实际数据, 而是存储查询的结构。每当用户查询视图时, 数据库会根据定义的查询动态生成结果。

视图通常用于将复杂的 SQL 查询封装成简单的视图 (**简化查询**), 隐藏复杂的数据结构, 让用户只能接触到必要的 (**数据抽象**), 并且可以限制用户只能查看视图中定义的数据, 避免直接访问底层数据表 (**数据安全**)。在航空公司航班查询系统的背景下, 我们可以根据数据需求设计几种不同类型的视图。

1.1.1 行列子集视图

行列子集视图是一个只选择数据库中部分行和部分列的视图。通过这个视图, 可以限制用户只能查看某些特定的列 (字段) 或者只返回符合特定条件的行。如果一个表中有很多列, 但只关心其中的几列或者某些特定的行数据, 可以通过行列子集视图来简化查询, 提高查询效率。

航班信息查询视图 根据 UI 界面需求, 需要一个显示航班信息的简化视图, 包括航班号、起降时间、出发和到达机场等字段。语句如下:

```
1 CREATE VIEW Flightsum AS
2 SELECT flight_number, departure_time, arrival_time, departure_airport
   , arrival_airport
3 FROM Flight;
```

用户历史航班视图 根据 UI 界面需求, 需要一个用户历史航班记录, 包括身份证号、姓名、航班号、起降时间。历史记录以 departur_time 字段判断, 小于即为历史记录。语句如下:

```
1 CREATE VIEW Passhistoryfli AS
2 SELECT id_card, passname,
3     (
4         SELECT flight_number
5         FROM Ticket
6         WHERE Ticket.id_card = Passenger.id_card
7     ) AS flight_number,
8     (
```

```

9      SELECT departure_time
10     FROM Flight
11    WHERE Flight.flight_number = (
12           SELECT flight_number
13           FROM Ticket
14           WHERE Ticket.id_card = Passenger.id_card
15       )
16      AND departure_time <= NOW()
17   ) AS departure_time,
18   (
19       SELECT arrival_time
20     FROM Flight
21    WHERE Flight.flight_number = (
22           SELECT flight_number
23           FROM Ticket
24           WHERE Ticket.id_card = Passenger.id_card
25       )
26      AND departure_time <= NOW()
27   ) AS arrival_time
28 FROM Passenger;

```

未来航班信息视图 根据 UI 界面需求，需要一个用户未来航班信息，包括身份证号、姓名、航班号、起降时间。历史记录以 `departur_time` 字段判断，大于即为未来。语句如下：

```

1 CREATE VIEW Passfuturefli AS
2 SELECT
3     id_card,
4     passname,
5     (
6         SELECT flight_number
7       FROM Ticket
8      WHERE
9         Ticket.id_card = Passenger.id_card
10    ) AS flight_number,
11    (
12        SELECT departure_time
13      FROM Flight
14     WHERE
15        Flight.flight_number = (
16            SELECT flight_number
17          FROM Ticket

```

```

18         WHERE
19             Ticket.id_card = Passenger.id_card
20     )
21     AND departure_time > NOW()
22 ) AS departure_time,
23 (
24     SELECT arrival_time
25     FROM Flight
26     WHERE
27         Flight.flight_number = (
28             SELECT flight_number
29             FROM Ticket
30             WHERE
31                 Ticket.id_card = Passenger.id_card
32         )
33     AND departure_time > NOW()
34 ) AS arrival_time,
35 (
36     SELECT airline_name
37     FROM Airline
38     WHERE
39         Airline.airline_code = (
40             SELECT airline_code
41             FROM Flight
42             WHERE
43                 Flight.flight_number = (
44                     SELECT flight_number
45                     FROM Ticket
46                     WHERE
47                         Ticket.id_card = Passenger.id_card
48                 )
49         )
50 ) AS airline_name
51 FROM Passenger;

```

未来航班信息视图 根据 UI 界面需求，需要一个个人信息视图，包括身份证号、姓名、年龄、身份证号。语句如下：

```

1 CREATE VIEW UserInfo AS
2 SELECT id_card, passname, age, phone
3 FROM Passenger;

```

1.1.2 带表达式的视图

带表达式的视图是在查询中使用了计算或表达式的视图。表达式可以是算术运算、字符串连接、日期操作、聚合函数等，它们能够在视图创建时直接计算结果，**适用于需要在查询时对数据进行计算或转换的场景。**

航班的飞行时长视图 根据 UI 界面需求，需要一个航班的航班号、起飞时间、降落时间与需要计算的飞行时长。。语句如下：

```
1 CREATE VIEW FlightTime AS
2 SELECT flight_number, departure_time, arrival_time,
3        TIMESTAMPDIFF(MINUTE, departure_time, arrival_time) AS
4        flight_time
5 FROM Flight;
```

1.1.3 分组视图

分组视图是通过 GROUP BY 子句来对查询结果进行分组的视图。它通常与聚合函数（如 SUM(), AVG(), COUNT() 等）一起使用，**用来展示每组数据的汇总信息。**如果需要对某些数据进行汇总统计，可以通过分组视图来实现。

航空公司下航班数量视图 根据 UI 界面需求，需要一个航空公司下航班数量视图，包括航班号、航空公司代码以及需要汇总的数字。语句如下：

```
1 CREATE VIEW Airlineflcount AS
2 SELECT airline_code, COUNT(*) AS flight_count
3 FROM Flight
4 GROUP BY airline_code;
```

1.2 测试视图

编写测试代码，通过视图实现数据查询操作

在上一小问中，创建了 6 个视图。对 6 个视图进行了查询简要航班信息、查询用户历史航班记录、查询用户未来航班信息、查询用户个人信息、查询航班的飞行时长、查询每个航空公司航班数量。数据查询操作如下：

```
1 -- 查询所有航班的简要信息
2 SELECT *
3 FROM Flightsum
4 WHERE departure_airport_name = 'PEK'
5        OR arrival_airport_name = 'PEK';
6
7 -- 查询某乘客的历史航班信息
8 SELECT * FROM Passhistoryfli WHERE id_card = '110101199001011234';
9
```

```

10 -- 查询某乘客的未来航班信息
11 SELECT * FROM Passfeturfli WHERE id_card = '110101199001011234';
12
13 -- 查询用户个人信息
14 SELECT * FROM UserInfo WHERE id_card = '110101199001011234';
15
16 -- 查询某航班的飞行时间
17 SELECT * FROM FlightTime WHERE flight_number = 'CA1234';
18
19 -- 查询每个航空公司航班数量
20 SELECT * FROM Airlineflicount;

```

查询结果如图4。

flight_number	departure_time	arrival_time	departure_airport_name	departure_city	arrival_airport_name	arrival_city
CA1112	2024-04-25 28:45:00	2024-04-25 23:45:00	PVG	Shanghai	PEK	Beijing
CA1234	2024-04-10 08:00:00	2024-04-10 10:30:00	PEK	Beijing	PVG	Shanghai
CA2849	2024-04-17 09:12:05	2024-04-17 13:12:05	PVG	Shanghai	PEK	Beijing
CA7836	2024-04-29 18:14:09	2024-04-29 22:14:09	PVG	Shanghai	PEK	Beijing
MU241	2024-04-29 17:01:06	2024-04-29 19:01:06	PVG	Shanghai	PEK	Beijing
MU678	2024-04-10 09:00:00	2024-04-10 11:45:00	PVG	Shanghai	PEK	Beijing
MU7290	2024-04-15 07:43:00	2024-04-15 11:43:00	PVG	Shanghai	PEK	Beijing

7 rows in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

id_card	passname	flight_number	departure_time	arrival_time
110101199001011234	Zhangsan	CA1234	2024-04-10 08:00:00	2024-04-10 10:30:00

1 row in set (0.01 sec)

Query OK, 0 rows affected (0.00 sec)

id_card	passname	flight_number	departure_time	arrival_time	airline_name
110101199001011234	Zhangsan	CA1234	NULL	NULL	Air China

1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

id_card	passname	age	phone
110101199001011234	Zhangsan	30	13800138000

1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

flight_number	departure_time	arrival_time	flight_time
CA1234	2024-04-10 08:00:00	2024-04-10 10:30:00	150

1 row in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

airline_code	flight_count
CA	4
MU	3

2 rows in set (0.00 sec)

图 4: 测试视图结果

1.3 修改视图

对不同类别视图尝试进行数据修改和删除操作，看能否实现，分析其原因

尝试修改行列子集视图 **UserInfo** 测试语句如下：

```

1 UPDATE UserInfo
2 SET phone = '19988887777'
3 WHERE id_card = '110101199001011234';

```

```
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

图 5: 修改行列子集视图 UserInfo 结果

成功，因为 UserInfo 是基于单表 Passenger 的简单视图，不涉及聚合或子查询。

尝试从历史航班视图删除记录 测试语句如下：

```
1 DELETE FROM Passhistoryfli WHERE id_card = '110101199001011234';
```

```
ERROR 1288 (HY000): The target table Passhistoryfli of the DELETE is not updatable
```

图 6: 从历史航班视图删除记录

失败，出现报错。原因是该视图含有嵌套 SELECT，并非直接映射真实表，MySQL 不允许从该视图进行修改/删除。

尝试修改 FlightTime 视图中的计算字段结果 测试语句如下：

```
1 UPDATE FlightTime
2 SET flight_time = 999
3 WHERE flight_number = 'CA1234';
```

```
ERROR 1348 (HY000): Column 'flight_time' is not updatable
```

图 7: 修改 FlightTime 视图中的计算字段结果

失败，出现报错。原因是该视图中包含了计算字段 TIMESTAMPDIFF(...)，此类派生列不可写。

尝试删除 Airlineflicount 分组视图中的记录 测试语句如下：

```
1 DELETE FROM Airlineflicount WHERE airline_code = 'CA';
```

```
ERROR 1288 (HY000): The target table Airlineflicount of the DELETE is not updatable
```

图 8: 删除 Airlineflicount 分组视图中的记录结果

失败，出现报错。原因是该视图是 GROUP BY 聚合视图，不可更新或删除。

修改删除视图总结 根据视图的性质，总结表格1如下。

视图名称	是否可修改/删除	原因说明
UserInfo	是	简单行列子集视图，基于单表 Passenger，结构简单，支持修改操作
Flightsum	否	虽为行列子集，但涉及多个外键列（机场信息），不建议直接修改
Passhistoryfli	否	视图中包含嵌套子查询，MySQL 不允许通过此类视图修改或删除
Passfuturefli	否	同样包含嵌套子查询与 NOW() 条件，结构复杂，不可更新
FlightTime	否	包含计算表达式 (TIMESTAMPDIFF)，为派生字段，不可直接更新
Airlineflicount	否	为 GROUP BY 分组视图，不支持数据更新或删除

表 1: 各类视图可修改性分析

1.4 视图作用

在实验报告中总结视图的主要作用。视图是一种基于查询语句生成的虚拟表，它本身不存储数据，而是通过对基础表的查询动态展示结果。我们在数据库结构搭建的基础上，设计了多种视图，用于满足用户界面所需的不同信息展示需求。

视图的一个显著作用是**简化复杂查询操作**。在这个航空业务中，用户往往需要同时查看来自多个表的信息，例如航班号、起降时间、机场名称和所属城市等。若每次都手动书写多表查询语句，不仅麻烦，而且容易出错。通过建立如 Flightsum 的视图，可以将这些查询逻辑封装在视图内部，前端应用或用户查询时只需操作一个结构清晰的“表”，大大提高了使用效率。

视图还能提升**数据的安全性**。**不是所有用户都应当拥有访问全部字段或修改底层数据表的权限**。通过创建只包含部分字段的视图，如 UserInfo，可以有选择地开放数据访问范围，配合数据库的权限管理机制，有效避免了数据误删、误改等问题。

在某些情况下，视图不仅可以查询，还支持**数据的更新操作**。如实验中对基于单表的 UserInfo 视图进行更新测试发现，该视图可以直接修改字段内容。这说明视图也可以用来构建更安全、更规范的数据修改入口，前提是满足 MySQL 对“可更新视图”的要求。

2 题目 2：用户口令管理设计题

用户注册与认证管理是数据库系统的基本功能，请为你在作业 2 中选择的具体案例设计一个用户注册与认证中的口令管理解决方案，要求如下

2.1 用户口令

用户口令采用密文存储，采用不可逆加密算法加密，查资料了解现有的不可逆算法。

在网络安全领域，用户口令通常通过**不可逆加密算法**（即哈希函数）进行存储，以防止明文密码泄露。不可逆哈希算法将明文口令转换为固定长度的散列值，保证单向不可逆，无法从散列值直接还原原始口令。这种机制使得即使攻击者获取了加密后的口令数据库，也难以还原出用户密码，提高了认证系统的安全性 [2]。然而，随着计算能力的提升和攻击技术的发展，传统的快速哈希算法在密码存储场景中逐渐暴露出安全隐患。

早期的口令存储广泛采用通用**快速哈希函数**，例如 MD5、SHA-1 和 SHA-256 等。这类算法计算高效，但并非专为密码学设计的“慢哈希”，其高速特性意味着攻击者可以在短时间内尝试大量候选密码，从而利用暴力破解和彩虹表等方式攻破口令 [2]。研究指出，这些快速哈希虽然具备抗碰撞等特性，但在抵御现代密码破解手段方面“几乎不提供保护”。为此，领域内引入了**加盐策略**，为每个口令附加一个随机盐值以抵御彩虹表攻击。同时，更为关键的是发展出**自适应口令哈希算法**，通过增加计算成本来降低攻击效率。

如 bcrypt 和 PBKDF2 就是这类算法的代表，其中 bcrypt 允许设置“工作因子”来调整计算难度，而 PBKDF2 通过高迭代次数增强安全性。然而，这些算法在 GPU 等高并行硬件上仍存在被加速破解的风险 [2]。为进一步提升安全性，研究者提出了**内存硬耗**哈希算法，如 scrypt 和 Argon2，它们通过强制使用大量内存资源限制攻击者的并行破解能力。其中，Argon2 自 2015 年在密码哈希竞赛中获胜后被广泛推荐，并发展出 Argon2d、Argon2i、Argon2id 等多个版本，以适应不同的安全需求 [1]。

在安全性比较方面，Argon2 被认为在抗 GPU/ASIC 攻击、抵御侧信道等方面均表现优异，是目前应用中安全性最高的算法之一。bcrypt 虽然已有多年应用历史，也具备良好的安全性，但在并行防御能力上略逊一筹。PBKDF2 则更适合对安全要求不极端苛刻但需要兼容性的场景，其安全性依赖于迭代次数的设置 [2]。而对于 SHA-256 等传统快速哈希算法，在用户口令存储中已被认为不安全，除非结合高强度密钥拉伸机制使用 [2]。

近年来，研究还引入了如 pepper 机制、非对称计算开销设计等新方法，用于进一步提高攻击者破解难度 [4]。同时，如何在不降低安全性的前提下提升算法在合法用户验证过程中的效率，也成为重要研究方向，例如在 GPU 上对 Argon2 的并行优化研究 [3]。

总的来说，用户口令加密算法正朝着更高安全性、更强硬件对抗能力方向发展。采用如 Argon2id 等新型、经安全验证的算法，并结合加盐、参数调整等策略，是当前用户认证系统设计中不可或缺的一部分。

2.2 密码修改

设计用户修改密码流程，画出流程图

流程图如图9所示。

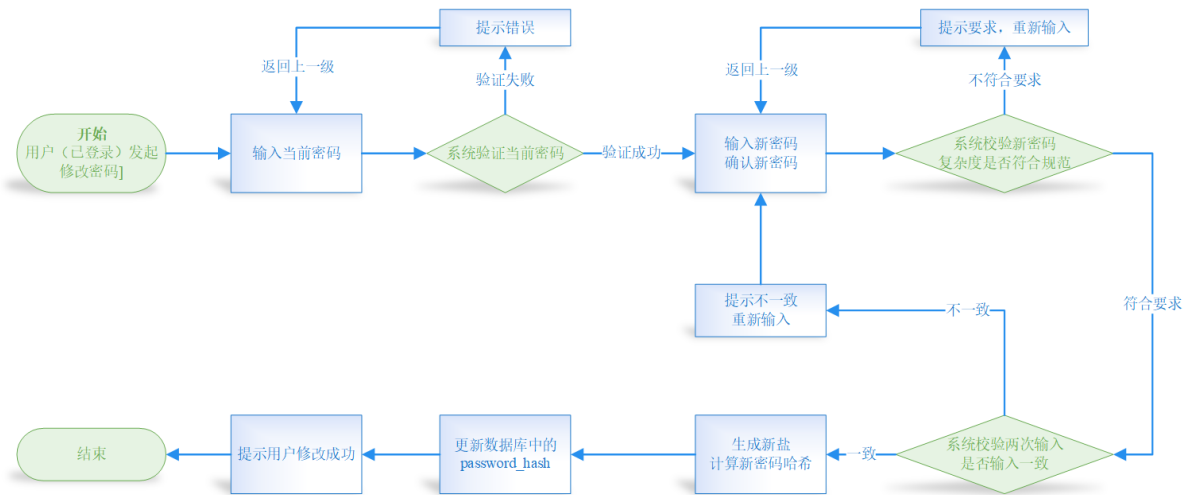


图 9: 密码修改流程

流程描述如下：

1. **用户发起请求：**用户在登录状态下，通过系统界面发起修改密码请求。
2. **系统验证当前密码：**
 - (a) 系统接收用户输入的当前密码。
 - (b) 按照登录流程，使用存储的 `password_hash` 中的盐和参数，对用户输入的当前密码进行哈希计算。
 - (c) 比较计算结果与存储的哈希值。
 - **验证失败：**提示用户“当前密码错误”，返回上一级。
 - **验证成功：**继续下一步。
3. **输入新密码：**系统提示用户输入新密码，并要求输入第二次以确认（防止输入错误）。
4. **新密码复杂度校验：**系统检查新密码是否符合预设的复杂度策略（如长度、包含大小写字母、数字、特殊字符等）。
 - **校验失败：**提示用户密码不符合要求，要求重新输入。
 - **校验成功：**继续下一步。
5. **确认新密码一致性：**比较两次输入的新密码是否完全相同。
 - **不一致：**提示用户两次输入不匹配，要求重新输入。
 - **一致：**继续下一步。
6. **生成新哈希：**
 - (a) 为新密码生成一个新的、唯一的随机盐。

(b) 使用与之前相同的算法（如 Argon2id）和配置参数，结合新盐，计算新密码的哈希值。

(c) 生成包含新算法标识、参数、新盐和新哈希值的完整字符串。

7. **更新数据库**：将新生成的 password_hash 字符串更新到该用户对应的 Passenger 表记录中。

8. **通知用户**：系统向用户显示“密码修改成功”的提示。

2.3 密码重置

设计用户密码重置业务流程（可以包含线下流程），画出业务流程图
流程图如图10所示。

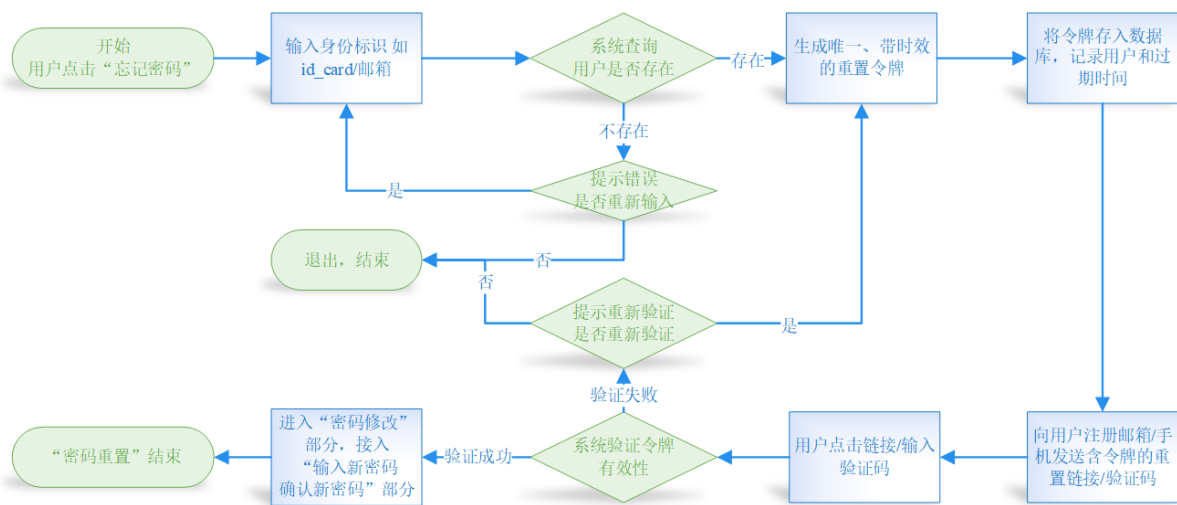


图 10: 密码重置流程

流程描述如下：

1. **用户发起请求**：用户在登录页面点击“忘记密码”链接。
2. **输入身份标识**：系统要求用户输入其注册时使用的身份标识（如身份证号 id_card、注册邮箱或手机号）。
3. **系统验证用户存在性**：系统根据用户输入的标识查询 Passenger 表，确认用户是否存在。
 - **用户不存在**：选择是否重新输入。如果是，则进入“输入身份标识”，反之结束流程。
 - **用户存在**：继续下一步，获取该用户的注册联系方式（预留电话/邮箱）。
4. **生成重置令牌**：系统生成一个唯一的、具有时效性（如 3 分钟）、与用户关联的密码重置令牌（Token）。此令牌应存储在数据库中，并记录生成时间和过期时间。

5. **发送重置链接/验证码**：系统向用户注册的邮箱或手机发送一封包含密码重置链接的邮件，或者包含验证码的短信。该链接应包含生成的重置令牌。
6. **用户操作**：用户检查邮箱（或手机），点击重置链接，或者页面输入验证码。
7. **系统验证令牌**：用户点击链接后，系统接收到请求中的令牌。系统查询数据库，验证令牌是否存在、是否与该用户关联、是否在有效期内。
 - **验证失败**（令牌无效/过期/错误）：显示错误页面，提示“链接已失效或无效，请重新申请”，选择是否继续流程。如果是，则进入“生成重置令牌”，反之结束流程。
 - **验证成功**：继续下一步。
8. **输入新密码**：系统展示一个页面，要求用户输入新密码，并确认新密码。进入上一部分“2.2 密码修改”。

由于此流程在目前现实生活中，线下流程几乎无需用到，因此此处忽略展示。

3 题目 3：用户授权操作题

为你在作业 2 中选择的具体案例增加若干个数据库用户，并对各个用户进行相应的授权操作，至少应完成如下操作：

1. 创建用户
2. 为不同的用户授予和收回表级 *SELECT, INSERT, UPDATE, DELETE, ALL PRIVILEGES* 权限
3. 为不同的用户授予和收回属性列级的 *SELECT, INSERT, UPDATE* 的权限
4. 创建角色，并把角色授予用户或收回用户的某个角色

将以上语句写成单独的 *SQL* 脚本，上机实验调试通过，编写测试代码，测试权限设置或取消是否成功，并将调试结果截屏写入实验报告。

3.1 用户与角色

用户 (User) 用户是数据库系统的认证主体。每个需要访问数据库的个人或应用程序都应该拥有一个唯一的用户账户。用户通过提供凭据（如用户名和密码）登录数据库。权限可以直接授予用户。

角色 (Role) 是一组权限的命名集合，可以把角色看作是一个“职位”或“职责分组”，这个职位自带一套操作数据库的权力。创建角色时，它本身没有任何权限，需要将具体的权限（如对某个表的 *SELECT*、*INSERT* 权限）授予给这个角色。角色不能直接登录数据库，它不是一个认证主体。

对于两者关系，管理员将执行特定任务所需的各种权限（如 SELECT、UPDATE、DELETE 等）打包授予给一个或多个角色。例如，可以创建一个 viewer 角色并授予它对所有表的 SELECT 权限。（**权限授予给角色**）管理员将一个或多个角色授予给用户。一个用户可以拥有多个角色，从而继承这些角色所包含的所有权限。例如，用户 *alice* 可以被授予 viewer 角色。（**角色授予给用户**）用户通过被授予的角色继承了该角色所拥有的所有权限。如果 *alice* 被授予 viewer 角色，她就自动获得了 viewer 角色被赋予的所有 SELECT 权限。（**权限继承**）

用户是登录数据库的实体，角色是权限的集合。可以与 C++ 进行类比，角色是权限的模板（class），用户是权限的使用者（object）。可以以下面的代码进行理解

```
1 CREATE ROLE 'developer';
2 GRANT SELECT, INSERT ON projectdb.* TO 'developer';
3
4 CREATE USER 'alice'@'localhost' IDENTIFIED BY 'password';
5 GRANT 'developer' TO 'alice'@'localhost';

1 class Developer {
2 public:
3     void select();
4     void insert();
5 };
6 Developer alice; // 相当于把角色 developer 授权给用户 alice
```

通过将角色授予用户，用户可以方便地获得执行其任务所需的一整套权限，大大简化了权限管理的复杂性。

3.2 用户语句

表级权限（Table-level Privileges）是对整个数据表的授权，用户在获得权限后可以对应表中所有列进行对应操作。一般用于一般业务用户，操作整表数据等。

属性列级权限（Column-level Privileges）是对某张表中指定的列进行授权，用户只能对被授权的字段进行操作，未授权的列将无法访问。一般用于敏感信息控制，如隐藏身份证、联系方式等。

在 MySQL 数据库中，**ALL PRIVILEGES** 是一个快捷方式，用于一次性授予某用户对某个数据库或表的所有可用权限。它等价于显式地授予 SELECT、INSERT、UPDATE、DELETE、CREATE、DROP、INDEX、ALTER、EXECUTE、REFERENCES 权限的组合。适用于数据库管理员或超级用户。

3.2.1 创建用户

语句如下：

```
1 DROP USER IF EXISTS 'alice'@'localhost';
2 DROP USER IF EXISTS 'bob'@'localhost';
3 DROP USER IF EXISTS 'robin'@'localhost';
```

```

4 -- 创建用户，设置密码
5 CREATE USER 'alice'@'localhost' IDENTIFIED BY '12345678';
6 CREATE USER 'bob'@'localhost' IDENTIFIED BY '12345678';
7 CREATE USER 'robin'@'localhost' IDENTIFIED BY '12345678';
8 -- 显示已创建的用户
9 SELECT user, host FROM mysql.user WHERE user IN ('alice', 'bob', '
    robin');
10 -- 授予 数据库级的 USAGE 权限，可以进入数据库
11 GRANT USAGE ON airlinedb.* TO 'alice'@'localhost';
12 GRANT USAGE ON airlinedb.* TO 'alice'@'localhost';
13 GRANT USAGE ON airlinedb.* TO 'alice'@'localhost';

```

测试的结果如图11所示。

```

Query OK, 0 rows affected (0.01 sec)
Query OK, 0 rows affected (0.00 sec)
Query OK, 0 rows affected (0.01 sec)
Query OK, 0 rows affected (0.01 sec)
Query OK, 0 rows affected (0.01 sec)
Query OK, 0 rows affected (0.01 sec)
Query OK, 0 rows affected (0.01 sec)

+-----+-----+
| user | host |
+-----+-----+
| alice | localhost |
| bob   | localhost |
| robin | localhost |
+-----+-----+
3 rows in set (0.00 sec)

```

图 11: 创建用户

在测试后期，测试相关用户权限时，进入用户界面，输入语句出现报错，如图12。由报错可知，必须加上可以访问数据库的权限 `USAGE ON airlinedb.*`，才能够进入数据库。

```

C:\Users\Robin>mysql -u alice -p
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 11
Server version: 9.0.1 MySQL Community Server - GPL

Copyright (c) 2000, 2024, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> USE airlinedb;
ERROR 1044 (42000): Access denied for user 'alice'@'localhost' to database 'airlinedb'
mysql> SELECT passname, phone FROM Passenger;
ERROR 1046 (3D000): No database selected
mysql> SELECT age FROM Passenger;
ERROR 1046 (3D000): No database selected

```

图 12: 用户无法进入 Airlinedb 数据库

3.2.2 授予和收回表级权限

alice: SELECT、INSERT 单授权收回测试 语句如下:

```
1 -- 授予表级 SELECT 权限
2 GRANT SELECT ON airlinedb.Airport TO 'alice'@'localhost';
3 -- 授予表级 INSERT 权限
4 GRANT INSERT ON airlinedb.Passenger TO 'alice'@'localhost';
5 -- 查看 alice 的当前权限
6 SHOW GRANTS FOR 'alice'@'localhost';
7
8 -- 撤销 alice 对 Passenger 表的 INSERT 权限
9 REVOKE INSERT ON airlinedb.Passenger FROM 'alice'@'localhost';
10 -- 查看 'alice' 的权限
11 SHOW GRANTS FOR 'alice'@'localhost';
```

测试的结果如图13所示。

```
Query OK, 0 rows affected (0.00 sec)
Query OK, 0 rows affected (0.00 sec)

+-----+
| Grants for alice@localhost |
+-----+
| GRANT USAGE ON *.* TO 'alice'@'localhost' |
| GRANT SELECT ON 'airlinedb`.`airport` TO 'alice'@'localhost' |
| GRANT INSERT ON 'airlinedb`.`passenger` TO 'alice'@'localhost' |
+-----+
3 rows in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

+-----+
| Grants for alice@localhost |
+-----+
| GRANT USAGE ON *.* TO 'alice'@'localhost' |
| GRANT SELECT ON 'airlinedb`.`airport` TO 'alice'@'localhost' |
+-----+
2 rows in set (0.00 sec)
```

图 13: alice: SELECT、INSERT 单授权收回测试

bob: SELECT, INSERT, UPDATE, DELETE 批量授权回收测试 语句如下:

```
1 -- 授予对 Ticket 表的所有基本权限
2 GRANT SELECT, INSERT, UPDATE, DELETE ON airlinedb.Ticket TO 'bob'@'localhost';
3 -- 查看 bob 的当前权限
4 SHOW GRANTS FOR 'bob'@'localhost';
5 -- 撤销 bob 对 Ticket 表的 DELETE 权限
6 REVOKE DELETE ON airlinedb.Ticket FROM 'bob'@'localhost';
7 -- 查看 bob 的权限
8 SHOW GRANTS FOR 'bob'@'localhost';
```

测试的结果如图14所示。

```
Query OK, 0 rows affected (0.00 sec)

+-----+
| Grants for bob@localhost |
+-----+
| GRANT USAGE ON *.* TO 'bob'@'localhost' |
| GRANT SELECT, INSERT, UPDATE, DELETE ON 'airlinedb`.`ticket` TO 'bob'@'localhost' |
+-----+
2 rows in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

+-----+
| Grants for bob@localhost |
+-----+
| GRANT USAGE ON *.* TO 'bob'@'localhost' |
| GRANT SELECT, INSERT, UPDATE ON 'airlinedb`.`ticket` TO 'bob'@'localhost' |
+-----+
2 rows in set (0.00 sec)
```

图 14: bob: SELECT, INSERT, UPDATE, DELETE 批量授权回收测试

robin: ALL PRIVILEGES 授权回收测试 语句如下:

```
1 -- 授予对 Flight 表的 ALL PRIVILEGES
2 GRANT ALL PRIVILEGES ON airlinedb.Flight TO 'robin'@'localhost';
3 -- 查看 robin 的权限
4 SHOW GRANTS FOR 'robin'@'localhost';
5
6 -- 撤销 robin 对 Flight 表的 UPDATE 权限
7 REVOKE UPDATE ON airlinedb.Flight FROM 'robin'@'localhost';
8 -- 查看 robin 的权限
9 SHOW GRANTS FOR 'robin'@'localhost';
```

测试的结果如图15所示。

```
Query OK, 0 rows affected (0.00 sec)

+-----+
| Grants for robin@localhost |
+-----+
| GRANT USAGE ON *.* TO 'robin'@'localhost' |
| GRANT ALL PRIVILEGES ON 'airlinedb`.`flight` TO 'robin'@'localhost' |
+-----+
2 rows in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

+-----+
| Grants for robin@localhost |
+-----+
| GRANT USAGE ON *.* TO 'robin'@'localhost' |
| GRANT SELECT, INSERT, DELETE, CREATE, DROP, REFERENCES, INDEX, ALTER, CREATE VIEW, SHOW VIEW, TRIGGER ON 'airlinedb`.`flight` TO 'robin'@'localhost' |
+-----+
2 rows in set (0.00 sec)
```

图 15: robin: ALL PRIVILEGES 授权回收测试

3.2.3 授予和收回属性列级的权限

语句如下:

```
1 -- 为用户 alice 操作
2 -- 授予 alice 对 Passenger 表的 'passname' 和 'phone' 列的 SELECT 权限
3 GRANT SELECT (passname, phone) ON airlinedb.Passenger TO 'alice'@'localhost';
```

```

4  -- 授予 alice 对 Passenger 表的 'phone' 列的 UPDATE 权限
5  GRANT UPDATE (phone) ON airlinedb.Passenger TO 'alice'@'localhost';
6  -- 查看 alice 的权限
7  SHOW GRANTS FOR 'alice'@'localhost';
8
9  -- 撤销 alice 权限
10 REVOKE SELECT (passname) ON airlinedb.Passenger FROM 'alice'@'
    localhost';
11 -- 查看 alice 的权限，验证撤销
12 SHOW GRANTS FOR 'alice'@'localhost';

```

测试如图16。

```

mysql> GRANT SELECT (passname, phone) ON airlinedb.Passenger TO 'alice'@'localhost';
Query OK, 0 rows affected (0.01 sec)

mysql> GRANT UPDATE (phone) ON airlinedb.Passenger TO 'alice'@'localhost';
Query OK, 0 rows affected (0.00 sec)

mysql> SHOW GRANTS FOR 'alice'@'localhost';
+-----+
| Grants for alice@localhost |
+-----+
| GRANT USAGE ON *.* TO 'alice'@'localhost' |
| GRANT SELECT ('passname', 'phone'), UPDATE ('phone') ON 'airlinedb`.`passenger` TO 'alice'@'localhost' |
| GRANT `viewer`@'localhost' TO 'alice'@'localhost' |
+-----+
3 rows in set (0.00 sec)

mysql> REVOKE SELECT (passname) ON airlinedb.Passenger FROM 'alice'@'localhost';
Query OK, 0 rows affected (0.01 sec)

mysql> SHOW GRANTS FOR 'alice'@'localhost';
+-----+
| Grants for alice@localhost |
+-----+
| GRANT USAGE ON *.* TO 'alice'@'localhost' |
| GRANT SELECT ('phone'), UPDATE ('phone') ON 'airlinedb`.`passenger` TO 'alice'@'localhost' |
| GRANT `viewer`@'localhost' TO 'alice'@'localhost' |
+-----+
3 rows in set (0.00 sec)

```

图 16: 授予和收回属性列级的权限

在当进入 alice 用户查询界面时，在未撤销 alice 用户权限时，输入了语句如下：

```

1  mysql -u alice -p

1  USE airlinedb;
2  SELECT passname, phone FROM Passenger;
3  SELECT age FROM Passenger;

```

测试如图18与17。这里可以看出授权了的 passname 与 phone 可以访问，但是 age 因为未被授权无法访问。

```

mysql> SELECT age FROM Passenger;
ERROR 1143 (42000): SELECT command denied to user 'alice'@'localhost' for column 'age' in table 'passenger'

```

图 17: alice 用户查询界面 1

```
mysql> SELECT passname, phone FROM Passenger;
+-----+-----+
| passname | phone |
+-----+-----+
| Robin    | 2130427306 |
| Rachel   | (695)853-1407x0976 |
| Mary     | 572.245.9752x801 |
+-----+-----+
```

图 18: alice 用户查询界面 2

当 passname 与 phone 被撤销权限时，测试如图19，未被授权无法访问。

```
mysql> SELECT passname, phone FROM Passenger;
ERROR 1143 (42000): SELECT command denied to user 'alice'@'localhost' for column 'passname' in table 'passenger'
```

图 19: alice 用户查询界面 3

3.3 角色语句

语句如下：

```
1 DROP ROLE IF EXISTS 'viewer'@'localhost';
2 DROP ROLE IF EXISTS 'ticket_editor'@'localhost';
3 DROP ROLE IF EXISTS 'admin_flight'@'localhost';
4
5 -- 创建角色
6 CREATE ROLE 'viewer'@'localhost';
7 CREATE ROLE 'ticket_editor'@'localhost';
8 CREATE ROLE 'admin_flight'@'localhost';
9
10 -- viewer: 只读
11 GRANT SELECT ON airlinedb.Airport TO 'viewer'@'localhost';
12 GRANT SELECT ON airlinedb.Airline TO 'viewer'@'localhost';
13 GRANT SELECT ON airlinedb.Flight TO 'viewer'@'localhost';
14 GRANT SELECT ON airlinedb.Passenger TO 'viewer'@'localhost';
15 GRANT SELECT ON airlinedb.Ticket TO 'viewer'@'localhost';
16 GRANT SELECT ON airlinedb.Orderinfo TO 'viewer'@'localhost';
17 GRANT SELECT ON airlinedb.Flightsum TO 'viewer'@'localhost';
18 GRANT SELECT ON airlinedb.Passhistoryfli TO 'viewer'@'localhost';
19 GRANT SELECT ON airlinedb.Passfeturefli TO 'viewer'@'localhost';
20 GRANT SELECT ON airlinedb.UserInfo TO 'viewer'@'localhost';
21 GRANT SELECT ON airlinedb.FlightTime TO 'viewer'@'localhost';
22 GRANT SELECT ON airlinedb.Airlineflicount TO 'viewer'@'localhost';
23
24 -- ticket_editor: 编辑订单、机票
25 GRANT SELECT, INSERT, UPDATE, DELETE ON airlinedb.Ticket TO '
    ticket_editor'@'localhost';
```

```

26 GRANT SELECT, INSERT, UPDATE, DELETE ON airlinedb.Orderinfo TO '
    ticket_editor'@'localhost';
27 GRANT SELECT ON airlinedb.Flight TO 'ticket_editor'@'localhost';
28 GRANT SELECT ON airlinedb.Passenger TO 'ticket_editor'@'localhost';
29 GRANT SELECT ON airlinedb.Airport TO 'ticket_editor'@'localhost';
30 GRANT SELECT ON airlinedb.Airline TO 'ticket_editor'@'localhost';
31
32 -- admin_flight: 操作航班相关表
33 GRANT SELECT, INSERT, UPDATE, DELETE ON airlinedb.Flight TO '
    admin_flight'@'localhost';
34 GRANT SELECT ON airlinedb.Airport TO 'admin_flight'@'localhost';
35 GRANT SELECT ON airlinedb.Airline TO 'admin_flight'@'localhost';
36
37 -- 将角色授予用户
38 -- 清理之前直接授予用户的权限
39 DROP USER IF EXISTS 'alice'@'localhost';
40 DROP USER IF EXISTS 'bob'@'localhost';
41 DROP USER IF EXISTS 'robin'@'localhost';
42 CREATE USER 'alice'@'localhost' IDENTIFIED BY '12345678';
43 CREATE USER 'bob'@'localhost' IDENTIFIED BY '12345678';
44 CREATE USER 'robin'@'localhost' IDENTIFIED BY '12345678';
45
46 -- 将 viewer 角色授予 alice
47 GRANT 'viewer'@'localhost' TO 'alice'@'localhost';
48 -- 将 ticket_editor 角色授予 bob
49 GRANT 'ticket_editor'@'localhost' TO 'bob'@'localhost';
50 -- 将 admin_flight 角色授予 robin
51 GRANT 'admin_flight'@'localhost' TO 'robin'@'localhost';
52 -- 同时将 viewer 角色也授予 robin (一个用户可以有多个角色)
53 GRANT 'viewer'@'localhost' TO 'robin'@'localhost';
54
55 -- 查看用户的权限, 现在应该包含角色的权限
56 SHOW GRANTS FOR 'alice'@'localhost';
57 SHOW GRANTS FOR 'bob'@'localhost';
58 SHOW GRANTS FOR 'robin'@'localhost';
59
60 -- 从用户收回角色
61 -- 从 robin 收回 viewer 角色
62 REVOKE 'viewer'@'localhost' FROM 'robin'@'localhost';
63 -- 查看 'robin' 的权限
64 SHOW GRANTS FOR 'robin'@'localhost';

```

测试的结果如图20所示，与语句逻辑一致。

```
+-----+
| Grants for alice@localhost |
+-----+
| GRANT USAGE ON *.* TO 'alice'@'localhost' |
| GRANT 'viewer'@'localhost' TO 'alice'@'localhost' |
+-----+
2 rows in set (0.00 sec)

+-----+
| Grants for bob@localhost |
+-----+
| GRANT USAGE ON *.* TO 'bob'@'localhost' |
| GRANT 'ticket_editor'@'localhost' TO 'bob'@'localhost' |
+-----+
2 rows in set (0.00 sec)

+-----+
| Grants for robin@localhost |
+-----+
| GRANT USAGE ON *.* TO 'robin'@'localhost' |
| GRANT 'admin_flight'@'localhost', 'viewer'@'localhost' TO 'robin'@'localhost' |
+-----+
2 rows in set (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

+-----+
| Grants for robin@localhost |
+-----+
| GRANT USAGE ON *.* TO 'robin'@'localhost' |
| GRANT 'admin_flight'@'localhost' TO 'robin'@'localhost' |
+-----+
2 rows in set (0.00 sec)
```

图 20: 角色语句

4 题目 4：数据库完整性实验题

完善作业 3 中你的数据模型完整性设计，模型中至少应体现如下完整性要求：

1. 实体完整性
2. 参照完整性
3. 用户定义的完整性
4. 对部分完整性约束进行命名
5. 创建域以实现某些属性内容完整性

将以上实现代码加入到作业 3 的脚本中并调试通过，用违反相应约束的语句尝试，以验证完整性约束实施效果。注意观察不同的 *RDBMS* 平台对完整性的支持程度。

4.1 验证三种完整性约束和约束与域的实现

对于完整性约束与域的实现，对部分代码进行了修改。修改部分代码如下：

```
1 -- 命名 CHECK 约束
2 CREATE TABLE Passenger (
3     id_card VARCHAR(20) PRIMARY KEY,
4     passname VARCHAR(50) NOT NULL,
5     age INT NOT NULL,
6     phone VARCHAR(100),
7     CONSTRAINT chk_age CHECK (age >= 0 AND age < 120)
```

```

8 );
9
10 -- 创建自定义域, CHECK近似实现
11 CREATE TABLE Orderinfo (
12     order_id INT AUTO_INCREMENT PRIMARY KEY,
13     id_card VARCHAR(20) NOT NULL,
14     moneys DECIMAL(10, 2) NOT NULL,
15     payment INT NOT NULL,
16     CONSTRAINT chk_payment CHECK (payment IN (0, 1)),
17     FOREIGN KEY (id_card) REFERENCES Passenger(id_card)
18 );

```

DOMAIN 类型允许创建自定义数据类型的模板, 是一种“带有约束的数据类型”。查询可知 MySQL 不支持 DOMAIN 类型。MySQL 不支持 DOMAIN, 是因为它的核心架构没有构建“类型抽象层”, 并优先追求灵活性和性能, 而非完整的 SQL 规范实现。对于需要类似功能的应用, MySQL 通常通过 ENUM、CHECK、存储过程等方式手动维护一致性。因此, 对于 Orderinfo 的实现, 使用了 CHECK 进行实现。

但是此处继续写一个适用与 PostgreSQL 的 DOMAIN 语句。

```

1 CREATE DOMAIN payment_status AS INT
2 CHECK (VALUE IN (0, 1));
3
4 CREATE TABLE Orderinfo (
5     order_id SERIAL PRIMARY KEY,
6     id_card VARCHAR(20),
7     moneys DECIMAL(10, 2) NOT NULL,
8     payment payment_status -- 使用 DOMAIN
9 );

```

4.1.1 实体完整性

实体完整性要求主键字段必须唯一, 且不能为空。测试语句如下:

```

1 -- 插入一个新机场
2 -- 预期结果插入正常
3 INSERT INTO Airport VALUES ('CAN', 'Guangzhou_Baiyun_International_
   Airport', 'Guangzhou', '4F');
4
5 -- 尝试插入与已有主键重复的机场
6 -- 预期结果 ERROR 1062 (23000): Duplicate entry 'PEK' for key '
   PRIMARY'
7 INSERT INTO Airport VALUES ('PEK', 'Some_Airport', 'AnyCity', '4F');

```

测试结果如图21, 与预期相符。

```
mysql> source D:\Learning_ComputerLanguages\learning_language_sql\1_Database\1_exp3\exp3_3.sql
Query OK, 0 rows affected (0.00 sec)

Query OK, 1 row affected (0.01 sec)

Query OK, 0 rows affected (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

ERROR 1062 (23000): Duplicate entry 'PEK' for key 'airport.PRIMARY'
```

图 21: 实体完整性

4.1.2 参照完整性

参照完整性要求子表中的外键值必须在其对应的父表中已存在。测试语句如下：

```
1 -- 以 CA、PEK、PVG 为外键的航班
2 -- 预期结果插入正常
3 INSERT INTO Flight VALUES ('CA9999', 'AirbusA330', '2024-04-12 10:00:00', '2024-04-12 13:00:00', 'PEK', 'PVG', 'CA');
4
5 -- 插入航班, airline_code 不存在
6 -- 预期结果 ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails
7 INSERT INTO Flight VALUES ('AB1234', 'Boeing777', '2024-05-01 10:00:00', '2024-05-01 12:00:00', 'PEK', 'PVG', 'AB');
```

测试结果如图22，与预期相符。

```
Query OK, 1 row affected (0.01 sec)

Query OK, 0 rows affected (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails ('airlinedb'.flight, CONSTRAINT flight_ibfk_3 FOREIGN KEY (airline_code) REFERENCES airline (airline_code))
```

图 22: 参照完整性

4.1.3 用户定义完整性

用户定义完整性要求字段值必须满足人为定义的规则（如年龄区间、支付状态只能是0或1）。测试语句如下：

```
1 -- 合法年龄
2 -- 预期结果插入正常
3 INSERT INTO Passenger VALUES ('110101199003035678', 'Wangwu', 25, '13700000000');
4 -- 合法支付状态
5 -- 预期结果插入正常
6 INSERT INTO Orderinfo (id_card, moneys, payment) VALUES ('110101199003035678', 999.99, 1);
7
```

```

8 -- 非法支付状态（只能是0或1）
9 -- 预期结果 Check constraint 'Orderinfo_chk_1' is violated
10 INSERT INTO Orderinfo (id_card, moneys, payment) VALUES ('
    110101199001011234', 88.88, 2);
11 -- 非法年龄（超过上限）
12 -- 预期结果 Check constraint 'Passenger_chk_1' is violated
13 INSERT INTO Passenger VALUES ('110101199004046789', 'ZhaoLiu', 150, '
    13600000000');

```

测试结果如图23，与预期相符。

```

Query OK, 1 row affected (0.01 sec)

Query OK, 0 rows affected (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

Query OK, 1 row affected (0.01 sec)

Query OK, 0 rows affected (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

ERROR 3819 (HY000): Check constraint 'orderinfo_chk_1' is violated.
Query OK, 0 rows affected (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

ERROR 3819 (HY000): Check constraint 'passenger_chk_1' is violated.

```

图 23: 用户定义完整性

4.2 不同的 RDBMS 平台对完整性的支持程度

为了测试不同的 RDBMS 平台对完整性的支持程度，本人安装了 SQL Server 2022 Developer（如图24）进行测试。

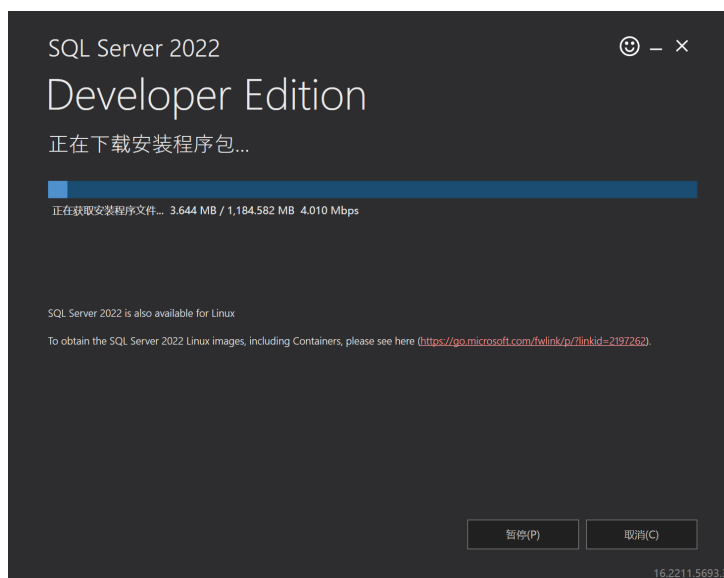


图 24: 安装 SQL Server

下载配置好后，本人输入以下语句，将 sql 语句载入到了 DBMS 中（如图25）。

```
1 sqlcmd -S localhost -U sa -P <密码> -i "D:\Learning_ComputerLanguages
   \learning_language_sql\1_Database\2_exp4\exp4SQLserver.sql"
2
3 sqlcmd -S localhost -U sa -P <密码> -i "D:\Learning_ComputerLanguages
   \learning_language_sql\1_Database\2_exp4\exp4_adddata.sql"
4
5 sqlcmd -S localhost -U sa -P <密码> -i "D:\Learning_ComputerLanguages
   \learning_language_sql\1_Database\2_exp4\exp4_adddata2.sql"
```

```
C:\Windows\System32>sqlcmd -S localhost -U sa -P [REDACTED] -i "D:\Learning_ComputerLanguages\learning_language_sql\1_Database\2_exp4\exp4SQLserver.sql"
已将数据库上下文更改为 "airlinedb"。
C:\Windows\System32>sqlcmd -S localhost -U sa -P [REDACTED] -d airlinedb -i "D:\Learning_ComputerLanguages\learning_language_sql\1_Database\2_exp4\exp4_adddata.sql"
(2 行受影响)
(2 行受影响)
(2 行受影响)
(2 行受影响)
(2 行受影响)
(2 行受影响)
C:\Windows\System32>sqlcmd -S localhost -U sa -P [REDACTED] -d airlinedb -i "D:\Learning_ComputerLanguages\learning_language_sql\1_Database\2_exp4\exp4_adddata2.sql"
(1 行受影响)
(1 行受影响)
```

图 25: sql 语句载入到 SQL Server

之后用先前语句进行测试（如图26）。

```
1> SELECT name FROM sys.tables;
2> GO
name
-----
Airport
Airline
Flight
Passenger
Ticket
Orderinfo
(6 行受影响)
1> SELECT *
2> FROM Flightsum
3> WHERE departure_airport_name = 'PEK'
4> OR arrival_airport_name = 'PEK';
5> GO
flight_number      departure_time      arrival_time      departure_airport_name departure_city      arrival_airport_name arrival_city
-----
CA1112             2024-04-25 20:45:09.000 2024-04-25 23:45:09.000 PVG                Shanghai           PEK                Beijing
CA1234             2024-04-10 08:00:00.000 2024-04-10 10:30:00.000 PEK                Beijing            PVG                Shanghai
CA2849             2024-04-17 09:12:05.000 2024-04-17 13:12:05.000 PVG                Shanghai           PEK                Beijing
CA7836             2024-04-29 18:14:09.000 2024-04-29 22:14:09.000 PVG                Shanghai           PEK                Beijing
MU4241             2024-04-29 17:01:06.000 2024-04-29 19:01:06.000 PVG                Shanghai           PEK                Beijing
MU5678             2024-04-10 09:00:00.000 2024-04-10 11:45:00.000 PVG                Shanghai           PEK                Beijing
MU7290             2024-04-15 07:43:00.000 2024-04-15 11:43:00.000 PVG                Shanghai           PEK                Beijing
(7 行受影响)
1> UPDATE UserInfo
2> SET phone = '19988887777'
3> WHERE id_card = '110101199001011234';
4> GO
(1 行受影响)
1> UPDATE FlightTime
2> SET flight_time = 999
3> WHERE flight_number = 'CA1234';
4> GO
消息 4406, 级别 16, 状态 1, 服务器 Robin, 第 1 行
对视图或函数 "FlightTime" 的更新或插入失败, 因其包含派生成或常量域。
1> SELECT * FROM UserInfo WHERE id_card = '110101199001011234';
2> GO
id_card            passname            age            phone
-----
110101199001011234  Zhangsan            30             19988887777
(1 行受影响)
1>
```

图 26: SQL Server 测试

对于 MySQL 与 SQL Server，在实体完整性方面，两者都提供了良好的支持。无论是主键、非空还是唯一性约束，两种平台都可以正常识别和执行。实验中，我在 MySQL 中

```
1> CREATE DOMAIN payment_status AS INTCHECK (VALUE IN (0, 1));
2> GO
消息 343, 级别 15, 状态 1, 服务器 Robin, 第 1 行
CREATE、DROP 或 ALTER 语句中使用了未知的对象类型 'DOMAIN'。
```

图 27: SQL Server DOMAIN

测试了对 Airport 表主键的重复插入，系统立即返回错误信息，表明实体完整性受到严格保护，SQL Server 在这方面亦同。在参照完整性方面，两者也都支持外键约束 (FOREIGN KEY)。但值得注意的是，MySQL 对外键的支持依赖于所使用的存储引擎——只有使用 InnoDB 引擎时，外键才会生效。而 SQL Server 原生支持外键，并且提供了更为灵活的设置，如 ON DELETE CASCADE 和 ON UPDATE CASCADE 等，在复杂数据关系维护中具有优势。在用户定义完整性方面，两者的差异较大。SQL Server 支持完整的 CHECK 约束机制、允许定义规则 (RULE) 等机制，MySQL 无法直接定义。但是两者都无法定义 DOMAIN (SQL Server 如图27)。

SQL Server 支持的触发器 (Trigger)、存储过程 (Stored Procedure) 和函数也更加灵活，有助于实现更高级的约束逻辑。虽然 MySQL 也支持触发器和存储过程，但在表达能力、异常处理和兼容性方面略显不足。

参考文献

- [1] Saran A N. On time-memory trade-offs for password hashing schemes[J]. *Frontiers in Computer Science*, 2024, 6: 1368362.
- [2] Dotsenko N, Chumachenko I, Galkin A, Chumachenko D. Password hashing methods and algorithms on the .NET platform[J]. *Advanced Information Systems*, 2024, 8(4): 82-92.
- [3] Eum S, Kim H, Song M, Seo H. Optimized Implementation of Argon2 Utilizing the Graphics Processing Unit[J]. *Applied Sciences*, 2023, 13(16): 9295.
- [4] Provos N. Bcrypt at 25: A Retrospective on Password Security[J]. *login.*, 2023.